

Secure License Manager - Complete Integration & Operations Manual

Multi-tenant, application-isolated, module-aware online licensing

2026-08-21

1 Integration guide

- 1.1 1. What the application integrates
- 1.2 2. Namespace model
- 1.3 3. Generate the application-scoped HWID
- 1.4 4. Create a persistent installation ID
- 1.5 5. Create the reusable client system
- 1.6 6. Gate application modules
- 1.7 7. Read all licensed modules
- 1.8 8. Refresh modules at runtime
- 1.9 9. Module changes
- 1.10 10. Ban/revoke behavior
- 1.11 11. Generic implementation contract for other languages
- 1.12 12. Server/admin provisioning flow
- 1.13 13. Recommended secure key storage
- 1.14 14. Production checklist

2 HWID specification

- 2.1 Security objective
- 2.2 Canonical algorithm
- 2.3 Stable OS machine component
- 2.4 Node.js SDK
- 2.5 C# derivation
- 2.6 Python derivation
- 2.7 Installation ID

3 TCP protocol

- 3.1 HELLO
- 3.2 WELCOME
- 3.3 HEARTBEAT
- 3.4 ENTITLEMENTS_REQUEST
- 3.5 LICENSE_EVENT
- 3.6 Module changes and key reissue

4 Admin REST API

- 4.1 Authentication
- 4.2 Vendors / tenants
- 4.3 Applications
- 4.4 Modules
- 4.5 Licenses
- 4.6 Clients
- 4.7 Dashboard and audit

5 Architecture

5.1 Security boundaries

5.2 Components

5.3 License JWT v2

5.4 Multi-tenant isolation rules

5.5 Modules

5.6 Horizontal scaling

6 Security model

6.1 Implemented controls

6.2 Critical integration rules

6.3 Production requirements

Protocol: Omni License Protocol v2

Stack: React + Node.js/TypeScript + Prisma ORM + PostgreSQL

Security: Ed25519 JWT, TLS 1.3/mTLS, application-scoped HWID, signed runtime entitlements

This manual is generated with the project and is intended for application developers, integrators, operators, and security reviewers. Public tenant/application UUIDs may be embedded in clients; server private signing keys and secrets must never be distributed.

1 Integration guide

1.1 1. What the application integrates

Every licensed application embeds only public configuration and client credentials:

- License server hostname/port.
- Vendor/tenant public UUID.
- Application public UUID.
- Ed25519 public license verification key.
- TLS CA trust anchor.
- Preferably a unique mTLS client certificate/key provisioned for that installation.
- The user's JWT license key, stored using OS-protected application storage when available.

Never embed the Ed25519 private signing key, HWID pepper, admin JWT secret, database password, CA private key, or admin credentials.

1.2 2. Namespace model

```
Tenant/Vendor A (UUID A)

  Application Desktop (UUID A1)

    CORE

    REPORTING

  Application Agent (UUID A2)

    CORE


Tenant/Vendor B (UUID B)

  Application Desktop (UUID B1)

    CORE

    REPORTING
```

Even when Vendor A and Vendor B sell applications with the same name/code, their tenant/application UUIDs are different. The client HWID, license JWT claims, database relationships, activation binding and signed events all include the namespace.

1.3 3. Generate the application-scoped HWID

Use the SDK rather than a global HWID:

```
const hwid = await generateHwid({ tenantId: TENANT_ID, applicationId: APPLICATION_ID });
```

On the same physical machine, `A/A1` , `A/A2` , and `B/B1` intentionally produce three different 64-character HWIDs.

1.4 4. Create a persistent installation ID

```
const clientId = await loadOrCreateScopedClientId(
  join(homedir(), '.my-product'),
  TENANT_ID,
  APPLICATION_ID
);
```

Persist this random UUID. Do not derive it from hardware.

1.5 5. Create the reusable client system

```
import { LicenseSystem, loadOrCreateScopedClientId } from '@license/node-sdk';

const license = new LicenseSystem({
  host: 'license.example.com',
  port: 7443,
  tenantId: TENANT_ID,
  applicationId: APPLICATION_ID,
  clientId,
  licenseKey: readLicenseFromSecureStorage(),
  caPath: './license-ca.pem',
  certPath: './client-cert.pem',
  keyPath: './client-key.pem',
  publicLicenseKeyPath: './license-ed25519-public.pem',
  expectedIssuer: 'secure-license-manager',
  expectedAudience: 'licensed-app',
  appVersion: APP_VERSION,
  platform: process.platform,
  onEntitlementsChanged(snapshot) {
    applyFeatureState(snapshot.modules);
  },
  onStateChange(event, reason) {
    if (['LICENSE_REVOKED', 'LICENSE_SUSPENDED', 'LICENSE_EXPIRED', 'CLIENT_BANNED', 'TENANT_SUSPENDED'].includes(event)) {
      disableLicensedFeatures(reason);
    }
    if (event === 'LICENSE_KEY_REISSUED') {
      showReplacementKeyRequiredMessage();
    }
  },
  onConnectionLost(error) {
    // Always-online policy: fail closed after validation timeout.
    disableLicensedFeatures(error.message);
  }
});

const entitlements = await license.connect();
```

`connect()` returns only after TLS authentication, JWT/database validation, namespace checks, activation/HWID checks, and module entitlement retrieval succeed.

1.6 6. Gate application modules

Do not parse the JWT in business logic. Use the validated session:

```
if (license.hasModule('REPORTING')) {  
  showReportingMenu();  
}  
  
license.requireModule('EXPORT_PDF');  
performPdfExport();
```

Use `requireModule()` again at sensitive execution boundaries, not only at menu rendering.

1.7 7. Read all licensed modules

For security, the server never authorizes or reveals a customer entitlement from HWID alone. The application sends its app-scoped HWID as one binding signal, but the module list is returned only after the JWT key, tenant/application namespace, database state, activation policy and TLS identity all validate.

```
const modules = license.getModules();  
const snapshot = license.getEntitlements();
```

A snapshot includes tenant ID, application ID, license ID, modules, entitlement version and validation timestamp.

1.8 8. Refresh modules at runtime

```
const fresh = await license.refreshEntitlements();  
console.log(fresh.modules);
```

The server returns an Ed25519-signed entitlement snapshot through the existing TCP channel. The SDK checks scope, sequence, expiration and signed module contents before replacing its cache.

1.9 9. Module changes

An administrator can change a license's selected modules. The manager increments `entitlementVersion`, reissues the license JWT, invalidates the old token hash for future connections, and pushes `LICENSE_KEY_REISSUED` to live activations.

Call `license.setLicenseKey(replacementJwt)` after securely receiving the new key, persist it, then reconnect when appropriate.

1.10 10. Ban/revoke behavior

- **Client ban:** blocks one installation/client UUID.
- **License revoke/suspend:** blocks every activation using that license.
- **Tenant suspend:** blocks every application/license below the tenant.
- **Expiry:** blocks the license after `expiresAt`.

The database is changed first; push notification is only the fast path. Offline clients are rejected on reconnect.

1.11 11. Generic implementation contract for other languages

A client in any language must implement this interface:

```
LicenseSystem.connect() -> EntitlementSnapshot  
  
LicenseSystem.close()  
  
LicenseSystem.isLicensed() -> bool  
  
LicenseSystem.getModules() -> string[]  
  
LicenseSystem.hasModule(code) -> bool  
  
LicenseSystem.requireModule(code) -> throws/denies  
  
LicenseSystem.refreshEntitlements() -> EntitlementSnapshot  
  
LicenseSystem.setLicenseKey(reissuedJwt)
```

Required internals:

1. TLS 1.3 server certificate validation; mTLS client certificate in production.
2. Application-scoped HWID v2 algorithm from `HWID-INTEGRATION.md`.
3. Persistent random installation UUID.
4. NDJSON protocol v2 HELLO/HEARTBEAT/ENTITLEMENTS_REQUEST.
5. Ed25519 verification of state/entitlement JWTs with exact issuer/audience.
6. Verify signed client ID, tenant UUID, application UUID, license ID, event, expiration and monotonic sequence.
7. Fail closed when heartbeat acknowledgements exceed the allowed validation timeout.
8. Never implement an arbitrary remote command handler.

1.12 12. Server/admin provisioning flow

1. Create Vendor/Tenant.
2. Copy its public tenant UUID.
3. Create Application inside that tenant.
4. Copy its public application UUID.
5. Add application Modules.
6. Generate License, selecting only authorized modules.
7. Deliver the one-time JWT to the customer's application securely.
8. Application generates scoped HWID and persistent client UUID, then connects.
9. Admin monitors activation, heartbeat and audit logs; ban/revoke if necessary.

1.13 13. Recommended secure key storage

- Windows: DPAPI / Credential Manager.
- macOS: Keychain.
- Linux desktop: Secret Service/libsecret where available; otherwise a root/user-protected file with restrictive permissions and disk encryption.
- Containers/services: secrets manager or orchestrator secret mount.

Never put the license JWT in command-line arguments, crash dumps, telemetry, analytics, URL query strings, or normal logs.

1.14 14. Production checklist

Use unique production secrets/certificates; enforce mTLS; protect signing keys with KMS/HSM; deploy admin HTTPS + SSO/MFA; use PostgreSQL TLS/least privilege/backups; add handshake rate limits; export audit logs to SIEM; add pub/sub before scaling TCP nodes; test ban/revoke/expiry/network-loss scenarios; fuzz the protocol; and perform independent security testing.

2 HWID specification

2.1 Security objective

The hardware identifier is a **binding signal**, not a password. Raw motherboard UUIDs, machine IDs, serial numbers, MAC addresses, and other hardware identifiers must never be sent to the License Manager.

Protocol v2 deliberately scopes the client HWID to both the **vendor/tenant UUID** and the **application UUID**. Therefore:

- same machine + same vendor + same application ⇒ stable same HWID;
- same machine + different application ⇒ different HWID;
- same machine + same application name/code under another vendor ⇒ different HWID;
- PostgreSQL stores neither the raw machine identifier nor the client-scoped HWID.

2.2 Canonical algorithm

First create an internal base fingerprint that is never transmitted:

```
components = [stable_os_machine_id, platform, cpu_arch]
canonical = SORT(LOWERCASE(TRIM(components))).JOIN("|")
base_hardware_fingerprint = HEX(SHA256("lm-base-hwid-v2|" + canonical))
```

Then derive the application-scoped value:

```
app_hwid = HEX(SHA256(
    "lm-app-hwid-v2|" +
    LOWERCASE(tenant_public_uuid) + "|" +
    LOWERCASE(application_public_uuid) + "|" +
    base_hardware_fingerprint
))
```

Only `app_hwid` is transmitted.

The server applies a second, secret HMAC layer before storage:

```
stored_hwid = HEX(HMAC_SHA256(
    HWID_PEPPER,
    "lm-server-hwid-v2|" + license_id + "|" + app_hwid
))
```

This creates three privacy/security boundaries: raw machine identifiers remain local; different applications cannot correlate the same computer by HWID; and a database leak does not expose the client-scoped HWID.

2.3 Stable OS machine component

- Windows: `Win32_ComputerSystemProduct.UUID`

- Linux: `/etc/machine-id`, fallback `/var/lib/dbus/machine-id`
- macOS: `IOPlatformUUID` from `IOPlatformExpertDevice`

Reject all-zero/all-F firmware placeholders. Do not use hostname or IP address as the primary identity.

2.4 Node.js SDK

```
import { generateHwid } from '@license/node-sdk';

const hwid = await generateHwid({
  tenantId: 'tenant-public-uuid',
  applicationId: 'application-public-uuid'
});
```

2.5 C# derivation

```
static string Sha256Hex(string value) =>
    Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(value)))
        .ToLowerInvariant();

var normalizedComponents = components
    .Select(x => x.Trim().ToLowerInvariant())
    .OrderBy(x => x, StringComparer.Ordinal);

var canonical = string.Join("|", normalizedComponents);
var baseFingerprint = Sha256Hex("lm-base-hwid-v2|" + canonical);
var appHwid = Sha256Hex(
    $"lm-app-hwid-v2|{tenantId.ToLowerInvariant()}|{applicationId.ToLowerInvariant()}|{baseFingerprint}");
```

2.6 Python derivation

```
import hashlib

def sha256_hex(value: str) -> str:
    return hashlib.sha256(value.encode("utf-8")).hexdigest()

canonical = "|".join(sorted(x.strip().lower() for x in components))
base_fingerprint = sha256_hex("lm-base-hwid-v2|" + canonical)
app_hwid = sha256_hex(
    f"lm-app-hwid-v2|{tenant_id.lower()}|{application_id.lower()}|{base_fingerprint}"
)
```

2.7 Installation ID

The HWID and installation/client ID solve different problems. Each application installation must also persist a random UUID. The Node helper creates an application-scoped storage filename:

```
const clientId = await loadOrCreateScopedClientId(
  appDataDirectory,
  TENANT_ID,
  APPLICATION_ID
);
```

Do not derive `clientId` from the HWID. A random persisted installation ID makes clone/reset workflows auditable and prevents one identifier from being overloaded for multiple security purposes.

3 TCP protocol

Protocol v2 uses one bidirectional **TLS 1.3** connection per running licensed application instance. Production deployments should require mTLS. The channel performs activation, continuous online validation, entitlement retrieval, heartbeat, and signed server-to-client license-state events. It is intentionally **not** a remote shell or arbitrary command channel.

Messages are UTF-8 newline-delimited JSON (NDJSON), maximum 64 KiB per frame.

3.1 HELLO

Immediately after TLS authentication:

```
{
  "type": "HELLO",
  "protocol": 2,
  "clientId": "persistent-random-installation-uuid",
  "licenseKey": "<JWT>",
  "tenantId": "tenant-public-uuid",
  "applicationId": "application-public-uuid",
  "hwid": "<64-char-application-scoped-sha256>",
  "appVersion": "2.4.0",
  "platform": "windows-x64"
}
```

The server rejects the connection unless all four scopes agree:

1. JWT `ten` claim equals the presented tenant UUID.
2. JWT `app` claim equals the presented application UUID.
3. Database license belongs to the same tenant/application.
4. HWID is the v2 fingerprint scoped to that tenant/application.

A valid key for one application therefore cannot be replayed by a different application on the same machine.

3.2 WELCOME

```
{
  "type": "WELCOME",
  "clientId": "...",
  "licenseId": "...",
  "state": "LICENSED",
  "tenantId": "...",
  "applicationId": "...",
  "modules": ["CORE", "REPORTING"],
  "entitlementVersion": 1,
  "heartbeatSeconds": 30,
  "serverTime": 1760000000000
}
```

`modules` is the authoritative feature set for the validated key. An application should enable gated functionality only after WELCOME has been accepted.

3.3 HEARTBEAT

```
{"type":"HEARTBEAT","ts":1760000000000,"nonce":"random-uuid"}
```

Response:

```
{"type":"HEARTBEAT_ACK","serverTime":1760000000000,"nonce":"same-nonce","entitlementVersion":1}
```

The default interval is 30 seconds; the server destroys stale sessions after 90 seconds. The reference SDK also uses a fail-closed acknowledgement watchdog.

3.4 ENTITLEMENTS_REQUEST

A running application can refresh its module list through the existing channel:

```
{"type":"ENTITLEMENTS_REQUEST","nonce":"random-uuid"}
```

Response:

```
{
  "type": "ENTITLEMENTS",
  "nonce": "same-nonce",
  "modules": ["CORE", "REPORTING"],
  "entitlementVersion": 1,
  "signedEvent": "<Ed25519 signed short-lived JWT>"
}
```

The client must verify that the signed snapshot matches the tenant UUID, application UUID, license ID, client ID, module list, entitlement version, issuer/audience, expiry, and monotonic sequence before applying it.

3.5 LICENSE_EVENT

```
{"type": "LICENSE_EVENT", "event": "LICENSE_REVOKED", "signedEvent": "<signed JWT>"}
```

Supported operational events include:

- LICENSE_REVOKED
- LICENSE_SUSPENDED
- LICENSE_EXPIRED
- LICENSE_RESTORED
- CLIENT_BANNED
- TENANT_SUSPENDED
- REVALIDATE_NOW
- LICENSE_KEY_REISSUED

State events contain tenant/application scope, license ID, client ID, sequence, module snapshot/entitlement version when available, short expiration, and an Ed25519 signature.

3.6 Module changes and key reissue

The module list is signed into the license JWT. Changing a license's selected modules increments `entitlementVersion` and returns a **replacement JWT**. The previous JWT hash is replaced in PostgreSQL, so the old key is rejected on reconnect. Connected clients receive `LICENSE_KEY_REISSUED` ; administrators must securely deliver/persist the replacement key in the licensed application.

4 Admin REST API

Base URL: `http://localhost:8080/api`

The browser admin uses a short-lived HttpOnly, SameSite session cookie with origin checks. All mutation endpoints require authenticated RBAC permissions.

4.1 Authentication

- `POST /auth/login`
- `GET /auth/me`
- `POST /auth/change-password`
- `POST /auth/logout`

4.2 Vendors / tenants

- `GET /tenants`
- `POST /tenants`

```
{ "code": "VENDOR_A", "name": "Vendor A", "description": "optional" }
```

Each tenant receives an immutable public UUID used as the protocol namespace.

- `POST /tenants/:id/status`

```
{ "status": "SUSPENDED" }
```

A suspended tenant causes all of its online licensing checks to fail closed.

4.3 Applications

Both `/applications` and the backward-compatible `/products` path address the application resource.

- `GET /applications`
- `POST /applications`

```
{  
  "tenantId": "database-tenant-id",  
  "code": "DESKTOP_PRO",  
  "name": "Desktop Pro",  
  "description": "optional"  
}
```

Each application receives a different public UUID even if two tenants use the same code/name.

4.4 Modules

- POST /applications/:applicationDatabaseId/modules

```
{"code":"REPORTING","name":"Reporting","description":"optional"}
```

- PATCH /applications/:applicationDatabaseId/modules/:moduleId

```
{"enabled":false}
```

A module that is still assigned to any license cannot be disabled; the API returns 409 until those licenses are reissued without that module.

4.5 Licenses

- GET /licenses?q=...
- POST /licenses

```
{
  "productId":"application-database-id",
  "moduleIds":["module-db-id-1","module-db-id-2"],
  "customerRef":"customer-123",
  "hwid":"optional-application-scoped-sha256-hwid",
  "expiresAt":"2027-08-21T00:00:00.000Z",
  "maxActivations":1,
  "metadata":{"edition":"pro"}
}
```

The plaintext JWT key is returned once and is not stored. PostgreSQL stores the token SHA-256 hash.

- POST /licenses/:id/modules

```
{"moduleIds":["module-db-id-1"]}
```

This reissues the JWT, increments entitlementVersion, invalidates the previous token hash, and returns the replacement key once.

- POST /licenses/:id/status

```
{"status":"REVOKED","reason":"Reported stolen"}
```

Valid states are ACTIVE, SUSPENDED, and REVOKED.

4.6 Clients

- GET /clients
- POST /clients/:clientId/revalidate

- `POST /clients/:clientId/ban`
- `POST /clients/:clientId/unban`

A client ban affects only that installation. License revocation affects all activations of the license. Tenant suspension affects every application/license below that tenant.

4.7 Dashboard and audit

- `GET /dashboard`
- `GET /audit`

5 Architecture

5.1 Security boundaries

```
Vendor / Tenant (public UUID)
└─ Application (different public UUID per tenant/application)
    ├── Module CORE
    ├── Module REPORTING
    └─ Module AUTOMATION
        └─ License (unique Ed25519 JWT + selected module set)
            └─ Activation (random client UUID + app-scoped HWID + mTLS fingerprint)
```

The same physical computer does not have one global License Manager HWID. It has a separate derived HWID for each (tenant UUID, application UUID) namespace. This intentionally prevents license reuse and cross-application correlation.

5.2 Components

```
React Admin Panel --HTTPS--> Node.js Admin API --Prisma--> PostgreSQL
|
+--- authoritative tenant/app/module/license state
|
Licensed application <--TLS 1.3/mTLS--> Node.js License TCP Server :7443
|
+-- LicenseSystem SDK          +-- signed validation/events
+-- app-scoped HWID            +-- in-memory live socket registry
+-- module gates
```

5.3 License JWT v2

Representative claims:

```
{
  "typ": "license",
  "jti": "license-id",
  "lic": "LIC-...",
  "ten": "tenant-public-uuid",
  "app": "application-public-uuid",
  "prd": "OMNI_APP",
  "mod": ["CORE", "REPORTING"],
  "ev": 1,
  "hwd": "optional-server-hmac-hwid",
  "iss": "secure-license-manager",
  "aud": "licensed-app"
}
```

The JWT proves issuance and immutable scope. PostgreSQL remains authoritative for revocation, suspension, current entitlement version, activation count, bans, and expiry.

5.4 Multi-tenant isolation rules

- Product/application code is unique **inside a tenant**, not globally.
- Public tenant UUID and application UUID are globally unique.
- Every HELLO contains both UUIDs.
- The JWT contains both UUIDs.
- The database relationship is checked again server-side.
- Application-scoped HWID includes both UUIDs.
- A persistent random client ID is checked against its existing license/HWID/application binding.
- An mTLS certificate fingerprint can be bound to the activation.

These checks are intentionally redundant so one compromised layer does not silently turn into cross-tenant license acceptance.

5.5 Modules

Modules are application-owned feature identifiers. A license selects zero or more enabled modules through `LicenseModule`. The authorized codes are signed into the JWT and returned in WELCOME and signed entitlement snapshots. Applications gate functionality with `hasModule()` / `requireModule()` rather than parsing the JWT themselves.

5.6 Horizontal scaling

The included live-client registry is in-memory and therefore single-node for immediate push delivery. PostgreSQL remains authoritative, so reconnect validation is safe. For multiple TCP nodes, use authenticated pub/sub (for example Redis or NATS) to route a ban/revoke/revalidation event to the process owning the socket, and use sticky/consistent routing where appropriate.

6 Security model

6.1 Implemented controls

- Ed25519 (`EdDSA`) allow-listed JWT signatures.
- Tenant and application UUIDs bound into license JWT claims and every client handshake.
- Module entitlement list and entitlement version signed into the license JWT.
- Application-scoped client HWID: same hardware produces different fingerprints for different tenant/application namespaces.
- Server-side HMAC-SHA256 of client HWID using a non-public pepper and license-specific context.
- Plaintext license JWT stored only at issuance response; PostgreSQL stores SHA-256 token hash.
- TLS 1.3 with optional/production-required mTLS.
- Activation-to-client UUID, HWID, tenant/application namespace and client-certificate fingerprint checks.
- Serializable activation transaction plus PostgreSQL advisory transaction lock for activation-count races.
- Always-online heartbeat and client-side validation watchdog.
- PostgreSQL-authoritative revoke/suspend/ban/expiry checks.
- Ed25519-signed, short-lived, sequenced state events and entitlement snapshots.
- No arbitrary server-to-client command execution.
- Argon2id admin password hashing, login throttling/lockout, RBAC, HttpOnly SameSite session cookies, origin checks, audit logs and token-version invalidation.
- Weak `admin / 123456` credential limited to an explicitly enabled development bootstrap and refused in production.

6.2 Critical integration rules

1. Never ship the Ed25519 **private** signing key, HWID pepper, admin JWT secret, database credentials, or CA private key in an application client.
2. Public tenant UUID, application UUID, server hostname, CA trust anchor/client certificate, and Ed25519 **public** verification key may be distributed as application configuration.
3. Do not unlock features by decoding the license JWT locally. Treat `LicenseSystem.connect()` / signed online entitlement state as the authorization source.
4. Deny licensed features when always-online validation exceeds the configured timeout.
5. Use `hasModule()` or `requireModule()` immediately before sensitive feature entry points; do not rely only on hiding a UI button.
6. Store a reissued JWT in OS-protected application storage where available. Never log it.
7. Treat HWID as non-secret and potentially clonable. It is one signal combined with client ID, online state, activation limits and preferably mTLS.

6.3 Production requirements

- Replace development certificates with a managed private PKI; issue a distinct client certificate per installation/device enrollment when practical.
- Protect signing material with KMS/HSM or a hardened signing service rather than a general application filesystem.
- Add signing-key rotation using `kid` with overlapping verification windows.
- Add SSO/MFA for administrators and remove password bootstrap after initial provisioning.
- Terminate the admin API behind HTTPS and a trusted reverse proxy/WAF; restrict TCP exposure to required networks where possible.
- Store secrets in a secrets manager and rotate them.
- Encrypt PostgreSQL at rest, enforce TLS to PostgreSQL, use least-privilege roles, backups and PITR.
- Export audit/security events to a SIEM; alert on repeated TLS rejects, activation-limit failures, cloned client IDs and unusual tenant/application mismatches.
- Rate-limit protocol handshakes at the network edge and enforce connection quotas.
- Add Redis/NATS authenticated pub/sub before horizontally scaling the TCP server.
- Perform dependency scanning, SAST/DAST, container scanning, penetration testing and protocol fuzzing before public deployment.